

PrivSplit: A Lossless Method for Prompt Privacy in Distributed Parameter-Efficient Fine-Tuning

Wujia Niu

University of Science and Technology of China
Hefei, Anhui, China
niuwjia@mail.ustc.edu.cn

Haoran Cheng

University of Science and Technology of China
Hefei, Anhui, China
chenghaoran@mail.ustc.edu.cn

Lan Zhang*

University of Science and Technology of China
Hefei, Anhui, China
zhanglan@ustc.edu.cn

Shen Li

University of Science and Technology of China
Hefei, Anhui, China
lishen02@mail.ustc.edu.cn

Abstract

Distributed Parameter-Efficient Fine-Tuning (PEFT) has emerged as a promising framework for personalizing Large Language Models (LLMs) by leveraging a collaboration between cloud servers and edge devices. However, this paradigm harbors a critical privacy vulnerability: the cloud can perform inversion attacks on the intermediate results (e.g., embeddings) sent from the edge to reconstruct the user's raw input prompt. Existing privacy-preserving techniques present a difficult trade-off: they are either too computationally expensive or they degrade model performance. To address this challenge, we introduce PrivSplit, a novel and lightweight protocol for prompt privacy in the Distributed PEFT framework. The core idea of PrivSplit is a "Split and Compensate" strategy: the edge device splits the input embedding into a public part sent to the cloud and a private part retained locally. The influence of this private part is then perfectly restored using a precise, lossless compensation mechanism during the interaction. We theoretically prove that PrivSplit is secure against both reconstruction and distinguishability attacks. Crucially, our method is perfectly lossless, ensuring that the model's performance is mathematically identical to the non-private baseline. This allows the security parameters to be arbitrarily strengthened without any impact on utility, breaking the conventional privacy-performance trade-off.

CCS Concepts

• Security and privacy → Privacy-preserving protocols; • Computing methodologies → Natural language processing.

Keywords

Privacy-Preserving Machine Learning; Large Language Models; Distributed Parameter-Efficient Fine-Tuning; Prompt Privacy

ACM Reference Format:

Wujia Niu, Lan Zhang, Haoran Cheng, and Shen Li. 2026. PrivSplit: A Lossless Method for Prompt Privacy in Distributed Parameter-Efficient

Fine-Tuning. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792112>

1 Introduction

Personalizing Large Language Models (LLMs) for individual users can greatly enhance their utility across diverse applications, but full-parameter fine-tuning is computationally expensive for models with billions of parameters. Parameter-Efficient Fine-Tuning (PEFT) addresses this limitation by updating only a small fraction of parameters, enabling efficient adaptation with significantly reduced resource requirements. However, the fine-tuning process itself requires a complete base model for computation. The immense memory and resource requirements of the base model make a purely edge-based deployment infeasible. Consequently, personalization is typically performed in a cloud-based setting, where a powerful server hosts the base model and applies user-specific PEFT modules. This cloud-centric paradigm introduces two critical challenges [11]: (1) uploading private data to the cloud for fine-tuning raises serious privacy concerns; and (2) maintaining a separate set of PEFT parameters for a large user base imposes considerable storage and management overhead, undermining scalability. Recent distributed PEFT [11, 33] frameworks mitigate these issues by retaining private data and lightweight PEFT modules (e.g., LoRA adapters) on edge devices while executing the computationally intensive base model in the cloud, exchanging only token embeddings or intermediate activations to enable scalable, privacy-preserving personalization.

New privacy threat. However, this advanced architecture persists with a critical privacy challenge. The standard distributed PEFT framework attempts to protect users' data privacy by having the edge device perform the initial token embedding and transmit only these vectors to the cloud [33]. Unfortunately, this approach is insufficient, as prior work has shown that the original text can be effectively reconstructed from its embeddings [17, 22, 28]. *This vulnerability is critically exacerbated in the distributed PEFT setting.* Because the cloud is the model provider, it possesses the token embedding matrix used on the edge. This reduces a complex optimization-based inversion attack to a trivial lookup. This lookup allows for a perfect 100% inversion of the original input prompt, creating an acute privacy risk.

Limitations of existing protections. Existing approaches for LLM prompt privacy can be classified into three main categories,



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates.*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792112>

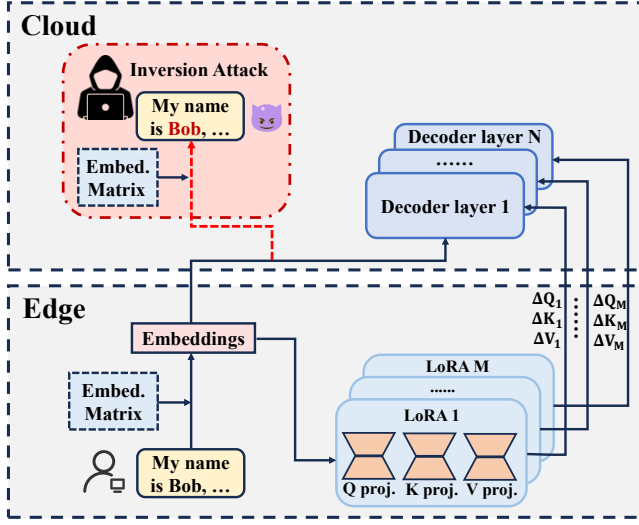


Figure 1: The standard insecure Distributed PEFT framework and its privacy vulnerability.

each presenting significant trade-offs [26] when considered for the distributed PEFT framework:

- (1) *Cryptography-based*. These approaches aim for strong security, ranging from heavy-weight protocols like Secure Multi-Party Computation (SMPC) [15, 16, 18] to light-weight token hashing and permutation schemes [21]. However, the former are impractical for real-time applications due to prohibitive computational overhead (e.g., several to over 100 seconds per token for a 7B model [35]), while the latter destroys the alignment between token embeddings and pre-trained model weights.
- (2) *Noise-based*. These approaches typically employ Differential Privacy (DP) [8] to add calibrated noise to input embeddings [7, 9, 20, 31, 34]. While providing formal guarantees, this noise injection inevitably degrades model performance, an issue that even auxiliary client-side denoising models [20] have struggled to fully resolve.
- (3) *Anonymization-based*. These approaches mask or replace sensitive entities [4, 37], but struggle with the difficulty of accurately identifying context-dependent information and often disrupt the semantic coherence required for high-performance reasoning.

In short, no existing approach offers a solution with strong privacy protection, lossless model utility, and low computational overhead for the distributed PEFT framework.

Our Approach. To bridge this gap, we propose *PrivSplit*, a novel protocol based on a “*Split and Compensate*” strategy. The process begins on the edge device by splitting the user’s input embedding into a public part, which is sent to the cloud, and a private part that is retained locally. Subsequently, the influence of this withheld private part is not simply discarded but is perfectly reintroduced. This is achieved via meticulously crafted compensation terms that the edge injects into the Query, Key, and Value (QKV) vectors during the collaborative interaction at the first LoRA-enabled decoder layer. The compensation is precisely calculated so that its effect manifests only after the self-attention and output projection steps, where it perfectly recombines with the public part

during the residual connection. This design ensures the resulting hidden state is mathematically identical to that of a non-private execution, guaranteeing perfectly lossless performance.

We validate the effectiveness of PrivSplit through rigorous theoretical analysis and comprehensive experiments. We first theoretically prove that the method is lossless. Our security analysis then formally establishes its resilience against two key threats: input reconstruction and input distinguishability. We prove that the difficulty of reconstructing the input approaches that of random guessing and that an attacker’s advantage in distinguishing two different inputs converges to zero as a user-controlled security parameter increases. Our extensive experiments conducted on multiple benchmarks corroborate these theoretical claims. For example, on the HellaSwag benchmark, PrivSplit achieves the same 75.9% accuracy as the non-private baseline, while comparable methods suffer a performance drop of over 17%. Against powerful inversion attacks, PrivSplit reduces the attack success rate to 0.01-0.06%. All of this is achieved with a negligible computational overhead of less than 1% of the total inference cost.

The main contributions of this work are summarized as follows:

- **A Principled Privacy-Preserving Framework.** We propose PrivSplit, a novel protocol that establishes a principled privacy-preserving framework for the distributed PEFT setting. To the best of our knowledge, it is the first method to provide lossless prompt privacy protection, achieving strong privacy guarantees and lightweight computations.
- **Rigorous Theoretical Guarantees.** We provide rigorous theoretical proofs for PrivSplit’s core properties. We formally prove that the method is lossless and establish its security guarantees against both input reconstruction and distinguishability attacks.
- **Extensive Experimental Validation.** We conduct comprehensive experiments on multiple benchmarks, demonstrating that PrivSplit maintains full model utility across various tasks and provides superior defense against attacks compared to existing methods.

2 Preliminary

2.1 Distributed Parameter-Efficient Fine-Tuning

Personalizing Large Language Models (LLMs) via Parameter-Efficient Fine-Tuning (PEFT) presents a significant deployment challenge, as a purely cloud-based approach raises privacy and scalability concerns [11], while an edge-only solution is often computationally infeasible on resource-constrained devices [3]. The distributed PEFT framework [11, 33] resolves this tension through a collaborative and heterogeneous architecture. In this paradigm, the cloud hosts the large, frozen base model, while the user’s edge device stores private data and lightweight, trainable modules (e.g., LoRA). The fine-tuning is then executed through an exchange of non-human-readable intermediate results, as detailed below.

Forward Propagation. The forward pass begins on the edge device, where an input text sequence is converted to numerical embeddings. To preserve data locality, only these embeddings are transmitted to the cloud. At each Transformer layer containing an active PEFT module, a collaborative computation occurs: the cloud

processes the incoming activations through its frozen blocks and sends the results back to the edge, which then performs its local forward pass through the trainable LoRA modules. This interactive process repeats for all layers equipped with active modules.

Backward Propagation. The backward pass mirrors this collaborative structure. After the training loss is computed, gradients are propagated backward through the frozen layers on the cloud. When the backward pass reaches a layer with an active PEFT module, the relevant gradients are sent to the edge device, which then uses them to perform its local backward propagation and update the weights of its LoRA modules.

2.2 Threat Model

We design PrivSplit to be secure under a specific and practical threat model that defines the adversary’s capabilities and goals, as well as the trusted components of the system.

Attacker Model. We consider the cloud service provider as the primary adversary. We model its behavior from two distinct aspects: its functionality and its security posture.

- **Functional Assumption (Honest):** The cloud is assumed to correctly follow the prescribed computational steps of the distributed PEFT framework, including the forward and backward propagation processes.
- **Security Assumption (Malicious):** From a privacy perspective, the cloud is considered malicious. It attempts to infer sensitive information from all intermediate results it observes. Furthermore, it will not cooperate with any extra privacy-enhancing requests from the edge device. The observable information includes the public portion of the user’s input embeddings and the compensated LoRA updates returned by the edge. We assume the cloud knows the base LLM’s architecture and weights and has full knowledge of the PrivSplit algorithm itself. However, it does not have access to the user’s raw data or their locally stored private LoRA parameters.

Privacy Goals. The attacker’s objective is to compromise the privacy of the user’s input prompt by analyzing the intermediate results it observes. This threat can manifest in several ways:

- **Input Reconstruction:** Recovering the original, raw text of the user’s prompt.
- **Membership Inference:** Determining whether a specific data point was included in the user’s fine-tuning dataset.
- **Attribute Inference:** Deducing sensitive attributes or the presence of specific keywords within the user’s prompt, even without recovering the full text.

System Assumptions. Our framework assumes that the user’s edge device is a trusted environment where computations can be performed securely. To execute the PrivSplit protocol for prompt privacy, the edge device requires the parameters of the LLM’s token embedding matrix and the first decoder layer. For scenarios that also require the protection of the model’s generated output (as discussed in Section 3.5), the edge device must additionally hold the parameters of the final decoder layer. The user’s private fine-tuning data and their personalized LoRA parameters are stored exclusively on the edge device and are never shared.

3 Methodology

3.1 The PrivSplit Mechanism

The core idea of PrivSplit is to protect the input prompt through a *Split and Compensate* strategy that is designed to be perfectly lossless. This strategy leverages the interactive nature of the distributed PEFT framework. The process begins on the trusted edge device by decomposing the input embedding into a public part for the cloud and a private part that is never transmitted. Instead of ignoring the withheld private part, its influence is strategically reintroduced during the collaborative computation at the first LoRA-enabled decoder layer. To achieve this, the edge injects meticulously crafted compensation terms into the Query, Key, and Value (QKV) projection vectors it returns to the cloud. This ensures the resulting hidden state is mathematically identical to that of a non-private execution, guaranteeing a perfectly lossless performance on the downstream task.

Following this core idea, we now present the detailed algorithmic derivation of PrivSplit, broken down into three parts. For clarity, we first illustrate the mechanism for a single-head attention model and discuss its extension to multi-head attention in Section 3.2.

Input Splitting. The process starts on the edge device. Let the initial input embedding on the edge device be $H_{in} \in \mathbb{R}^{L \times d}$, where L is the sequence length and d is the hidden dimension. The edge device decomposes this embedding into two components: $H_{in} = H_{e_in} + H_{p_in}$, where H_{e_in} is the public part sent to the cloud, and H_{p_in} is the private part retained locally. The cloud then proceeds with its initial computations on the public part, which includes a LayerNorm operation to produce $H_e = \text{LayerNorm}(H_{e_in})$.

Compensated LoRA Update Generation. The core of our mechanism occurs during the first-layer LoRA interaction. The cloud requests the LoRA computation from the edge by sending H_e . The edge’s task is to generate updates that compensate for the missing H_{p_in} . It first calculates the full normalized hidden state $H = \text{LayerNorm}(H_{in})$ and uses it to compute the compensated updates for the cloud’s QKV projections (H_{Q_cloud} , etc.). For the query and key vectors, the edge computes:

$$H_{Q_edge} = H(Q_{base} + Q_{lora}) - H_{Q_cloud} \quad (1)$$

$$H_{K_edge} = H(K_{base} + K_{lora}) - H_{K_cloud} \quad (2)$$

For the value vector, the compensation includes a crucial additional term, $\Delta V_L \in \mathbb{R}^{L \times d}$, designed to reintroduce H_{p_in} after the self-attention step:

$$H_{V_edge} = H(V_{base} + V_{lora}) - H_{V_cloud} + \Delta V_L \quad (3)$$

Pre-computation of the Compensation Term. The special term ΔV_L is constructed to satisfy the condition $A_L \Delta V_L O_{base} = H_{p_in}$, where A_L is the attention score matrix and O_{base} is the output projection matrix. To make this practical and avoid dependencies on the input-specific A_L , we construct a *data-independent* ΔV_L by setting $\Delta V_L = \mathbf{1}_L \cdot \Delta v^T$. Since the rows of the softmax-produced attention matrix A_L sum to one, the product $A_L \Delta V_L$ simplifies to a matrix where every row is the vector Δv^T . The condition thus simplifies to finding a vector Δv such that $\Delta v^T O_{base} = h_{p_in}^T$, where $h_{p_in}^T$ is a private vector applied to all token positions. The edge can pre-compute pairs of $(\Delta v, h_{p_in})$ offline for use during runtime.

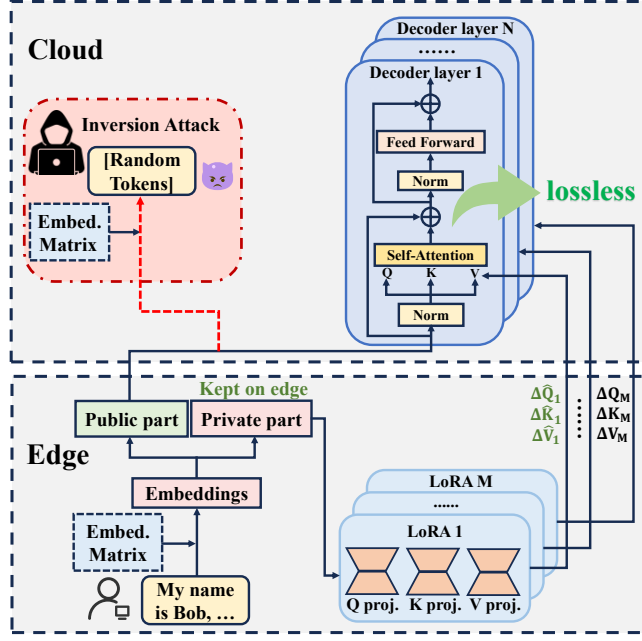


Figure 2: The architecture of PrivSplit. Notations with a hat (e.g., $\Delta\hat{Q}_1$) represent the compensated LoRA updates, distinguishing them from the standard ones (e.g., ΔQ_M).

Having detailed the compensation mechanism, we now present the following theorem to formally establish its lossless property.

THEOREM 1 (LOSSLESS PROPERTY). *The hidden state H_{MLP_in} at the input of the first MLP block in the PrivSplit protocol is mathematically identical to that of a non-private execution.*

PROOF. The cloud combines its projections with the edge's updates. Since the resulting Q and K vectors are exact, the attention matrix A_L is identical to the non-private case. The output of the self-attention module is then:

$$\begin{aligned} H_{SA_out} &= A_L(H(V_{base} + V_{lora}) + \Delta V_L)O_{base} \\ &= \underbrace{A_L H(V_{base} + V_{lora})O_{base}}_{\text{Standard SA Output}} + \underbrace{A_L \Delta V_L O_{base}}_{H_{p_in}} \end{aligned}$$

Finally, this output is added to the layer's initial input during the residual connection. Since the cloud uses the public part H_{e_in} , the input to the subsequent MLP block becomes:

$$\begin{aligned} H_{MLP_in} &= H_{e_in} + H_{SA_out} \\ &= H_{e_in} + (\text{Standard SA Output} + H_{p_in}) \\ &= \underbrace{(H_{e_in} + H_{p_in})}_{H_{in}} + \text{Standard SA Output} \end{aligned}$$

This result is mathematically identical to the MLP input in a non-private execution, proving that the PrivSplit method is lossless. $\square$

The complete end-to-end workflow of our method is formalized in a protocol with pseudocode in Appendix A.

3.2 Practical Considerations

For clarity, we have presented the core PrivSplit mechanism in an idealized context. To ensure its applicability in real-world scenarios, we must address two key practical considerations: its generalization to standard model architectures and its robustness in typical training and inference pipelines. Specifically, we first discuss how the mechanism extends to the prevalent multi-head attention architecture. Second, we analyze and provide a solution for a potential side-channel vulnerability introduced by the common practice of padding inputs in a batch.

Extension to Multi-Head Attention. The PrivSplit mechanism directly generalizes to the Multi-Head Attention (MHA) architecture standard in modern transformers while maintaining its lossless property. The compensation principle is applied independently to the Value projection of each attention head, and the overall guarantee is upheld after the outputs of all heads are concatenated and passed through the final output projection matrix. By applying the same data-independent construction for each head's compensation term, letting $\Delta V_i = \mathbf{1} \cdot v_i^T$ for the i -th head, the condition for perfect compensation simplifies to solving the linear system $[v_1^T, v_2^T, \dots, v_N^T] W_o^{(1)} = h_{p_in}^T$, where $[v_1^T, v_2^T, \dots, v_N^T]$ is the concatenation of the per-head compensation vectors. This presents the same fundamental problem as in the single-head scenario, allowing the same offline pre-computation strategy to be used. Given this direct generalization, the unified protocol presented in Algorithm 1 is sufficient for both architectures.

Handling Batched Inputs and Padding. A practical implementation must handle batched inputs, which are typically padded to a uniform length. This introduces a potential side-channel vulnerability, as the cloud can identify repeating embeddings of padding tokens and leverage this pattern to infer the private compensation part H_{p_in} . To mitigate this, our primary strategy is to assign different compensation vectors to the padding tokens than to the actual content tokens. This is made possible by the block structure of the combined attention mask used in padded sequences, which effectively decouples the computations for padded positions from those for content positions. Consequently, the compensation for the content part can maintain its data-independent structure as previously described, while the compensation for the padding part can be varied (e.g., randomized) to break the side-channel pattern without affecting the lossless property for the actual input. The formal derivation for this strategy is detailed in Appendix B.1.

3.3 Security Analysis

We analyze the security of PrivSplit against two fundamental types of attacks: input reconstruction attack and input distinguishability attack. Our analysis is based on a set of standard assumptions: from the cloud's perspective, the edge's unknown LoRA weights are modeled as random variables with each element drawn from an i.i.d. Gaussian distribution $\mathcal{N}(0, \sigma^2)$. Similarly, the private vectors $\Delta v_i, \Delta v$ used for compensation are sampled from an i.i.d. Gaussian distribution $\mathcal{N}(0, \sigma_\Delta^2 \mathbf{I})$, where σ_Δ^2 is a security parameter controlled by the edge.

We first consider the strongest possible attack, where the malicious cloud attempts to reconstruct the user's original input X from its set of observations $C = \{H_{e_in}, H_{Q_edge}, H_{K_edge}, H_{V_edge}\}$.

We quantify the difficulty of this attack using conditional entropy, $\mathcal{H}(X|C)$. A large value indicates that the cloud's observations provide little information about the secret input, making reconstruction difficult. Our analysis shows that the randomness of the unknown LoRA weights and the private compensation vectors effectively masks the secret input. We formalize this in the following theorem.

THEOREM 2. *The difficulty of the cloud to reconstruct the original input X from its observations C is fundamentally lower-bounded, preserving the vast majority of the input's uncertainty. Formally, this is expressed as a tight lower bound:*

$$\mathcal{H}(X|C) \geq \mathcal{H}(X) - \epsilon$$

where the total information leakage $\epsilon = \mathcal{I}(X; C)$ is a constant that is small relative to the total entropy $\mathcal{H}(X)$.

PROOF SKETCH. The detailed proof is provided in Appendix B.2. The core idea is to show that the mutual information $\mathcal{I}(H; C)$ between the secret normalized hidden state H and the cloud observations C is negligible. We prove this by individually bounding the information leakage from each observable component in C . The randomness introduced by the high-variance private part H_{p_in} and the unknown LoRA weights ensures that each observable component leaks a vanishingly small amount of information about H . $\square$

Next, we analyze a weaker but fundamental attack, where the adversary's goal is merely to distinguish between two different inputs, $X^{(0)}$ and $X^{(1)}$, with a probability better than random guessing. We measure the success of any such attack using the attacker's advantage, $\text{Adv}(\mathcal{A})^1$, which must be close to zero for the protocol to be secure.

THEOREM 3. *For any two distinct inputs $X^{(0)}$ and $X^{(1)}$, the advantage $\text{Adv}(\mathcal{A})$ for any computationally unbounded attacker $\mathcal{A}$ to distinguish between them based on the cloud's observations C vanishes as the security parameter σ_Δ^2 increases, i.e.*

$$\lim_{\sigma_\Delta^2 \rightarrow \infty} \text{Adv}(\mathcal{A}) = 0$$

implying that the cloud's observations for the two inputs become statistically indistinguishable.

PROOF SKETCH. The full proof is in Appendix B.3. The strategy is to show that the statistical distributions of the cloud observations for the two inputs, $P_{C^{(0)}}$ and $P_{C^{(1)}}$, become indistinguishable. We first relate the attacker's advantage to the Total Variation Distance between these two distributions. Using Pinsker's inequality, we then bound this distance with the KL divergence $D_{KL}(P_{C^{(0)}} || P_{C^{(1)}})$. The main part of the proof is to show that as the variance of the private compensation vectors (σ_Δ^2) grows, the noise completely masks the differences in the original inputs, causing the KL divergence and therefore the attacker's advantage to converge to zero. $\square$

In summary, our theoretical analysis demonstrates that the security of PrivSplit against both strong reconstruction and weaker distinguishability attacks is formally guaranteed. The security level can be arbitrarily improved by increasing the variance of the private

compensation vectors, controlled by the security parameter σ_Δ^2 . A key advantage of our provably lossless design is that the security parameter can be made arbitrarily large to achieve any desired level of privacy. Unlike noise-based privacy methods where a larger noise variance inevitably degrades model utility, the randomness introduced in our compensation scheme is perfectly canceled out during the computational process. This allows PrivSplit to break the conventional trade-off between privacy and performance, enabling users to configure a high level of security without any cost to the model's accuracy.

3.4 Overhead Analysis

We analyze the overhead introduced by PrivSplit in terms of both communication and computation.

Communication Overhead. The PrivSplit protocol is designed to be perfectly compatible with the standard distributed PEFT communication pattern. The tensors exchanged between the edge and cloud are identical in size to their counterparts in the non-private protocol. Therefore, PrivSplit introduces *zero additional communication overhead*.

Computational Overhead. Our protocol is transparent to the cloud provider, resulting in zero additional computational overhead on the cloud server. The computational overhead is exclusively localized on the edge device and occurs only once per inference, during the interaction at the first LoRA-enabled decoder layer. To quantify this cost, let L be the input sequence length and d be the hidden dimension of the model. The additional work required by PrivSplit is dominated by three matrix multiplications between the full hidden state (size $L \times d$) and the projection matrices of the base model $d \times d$ for Q, K and V . Each of these multiplications requires approximately $2Ld^2$ FLOPs, leading to a total extra cost of $C_{extra} \approx 6Ld^2$ FLOPs.

To evaluate the significance of this one-time cost, we compare it with the total computation of a full forward pass. For a decoder-only LLM with N layers, the total inference FLOPs, primarily from self-attention and FFN blocks in each layer, can be approximated as $C_{total} \approx 24N L d^2$ (assuming a multiply-add operation counts as 2 FLOPs). Therefore, the fraction of the total inference cost incurred as extra overhead on the edge is:

$$\frac{C_{extra}}{C_{total}} \approx \frac{6Ld^2}{24N L d^2} = \frac{1}{4N}$$

This result demonstrates that the overhead of PrivSplit is a small constant fraction dependent only on the model depth. For a 32-layer model like LLaMA-7B, this overhead is less than 1% of the total inference computation. This analysis confirms that the cost of PrivSplit is negligible when compared to the full model's computational scale, imposing an acceptable burden on the edge device.

3.5 Extension: PrivSplit with Output Privacy

Motivation. The PrivSplit mechanism detailed thus far provides robust protection for the user's input prompt. However, in many real-world interactive applications, the privacy considerations extend beyond the initial input to include the model's generated output. Consider a long-form dialogue with an LLM about sensitive topics such as personal health or confidential business strategy.

¹The attacker's advantage is formally defined as $\text{Adv}(\mathcal{A}) = |\Pr[b' = b] - 1/2|$, where b is the randomly chosen bit (0 or 1) corresponding to the input, and b' is the attacker's guess.

The model's responses are contextually entangled with the user's private information. An output like "Further analysis of the patient's symptoms suggests..." could indirectly reveal the nature of the user's private query to the cloud server, which orchestrates the generation. As the generated response becomes part of the ongoing conversation history, protecting its content is crucial for maintaining end-to-end privacy. This contrasts sharply with tasks such as classification, where the model's output is often a single, non-sensitive public label (e.g., "Positive"), the observation of which poses a negligible privacy risk. Therefore, to support a broader range of sensitive, interactive use cases, it is necessary to extend our framework also to protect the privacy of the generated output.

Architectural Changes. Protecting the generated output requires a key architectural modification: the LLM's final decoder layer and final classification layer (LM Head) must be offloaded from the cloud to the edge device. Furthermore, it is essential that a LoRA module is deployed on this final decoder layer. The rationale for this is two-fold. First, the output of the final decoder layer is the direct precursor to the generated token. Localizing this computation prevents the cloud from observing this state and using its own public LM Head to infer the model's output, thus preventing a direct privacy breach. Second, the edge-exclusive LoRA module is critical for preventing an inference attack. Without it, the cloud, which possesses the base model parameters, could take the $H_{out}^{(N-1)}$ state it generates and locally execute its own copy of the final decoder layer, thereby replicating the final hidden state. The private LoRA module ensures the final layer's transformation is unique to the edge device and computationally unreproducible by the cloud.

Algorithm Modification. We refer to this extended protocol as PrivSplit-OP (Output Privacy). The workflow under PrivSplit-OP diverges from the core protocol at the final stage of the forward pass. After the collaborative computation for the second-to-last layer is complete, the cloud's role in the forward pass concludes. It sends its final output, the hidden state $H_{out}^{(N-1)}$, back to the edge device. From this point, the edge takes over all subsequent computations entirely locally. It feeds $H_{out}^{(N-1)}$ into its local final decoder layer, which applies both the base and its private LoRA transformations to compute the final hidden state $H_{out}^{(N)}$. Finally, the edge applies its local LM Head to $H_{out}^{(N)}$ to generate the logits and sample the next token. This strict localization of the final computational steps ensures that the sensitive final hidden state and the resulting generated content are never exposed to the server.

Analysis of the Extended Model. The extended PrivSplit-OP protocol preserves the core lossless and input privacy guarantees of PrivSplit while introducing predictable and quantifiable overheads.

- **Losslessness and Security.** The protocol remains lossless, as offloading the final layer's computation to the edge does not alter its mathematical outcome. Furthermore, PrivSplit-OP inherits the strong prompt privacy guarantees from the core mechanism. By design, through the localization of the final computational steps, it also provides a guarantee for output privacy, as the cloud never observes the final hidden state or the resulting logits.
- **Communication Overhead.** The cloud-to-edge communication volume per generated token increases. While both protocols involve exchanging hidden states for LoRA layer computations,

their final transmission step is critically different. In the core PrivSplit protocol, the cloud's final transmission is a single token ID (e.g., 4 bytes). In PrivSplit-OP, this lightweight result is replaced by the transmission of the full hidden state from the second-to-last layer $H_{out}^{(N-1)}$, a tensor of size d (e.g., 16 KB for $d = 4096$). This represents a substantial increase in communication volume for each token generated.

- **Computational Overhead.** The computational burden on the edge device also increases. For each generated token, the edge is responsible for the full forward pass of the final decoder layer (approximately $24d^2$ FLOPs) and the final LM Head projection (approximately $2dV$ FLOPs, where V is the vocabulary size). This represents a clear trade-off for users requiring maximal privacy.

4 Experiments

4.1 Experimental Setup

Model and PEFT Configuration. All experiments are conducted using LLaMA2-7B [32] as the cloud-side base model. On the edge device, we employ LoRA as the PEFT method. Specifically, we apply LoRA modules to the query, key, and value projection matrices in the first and last decoder layers, with a rank of $r = 128$. This results in approximately 6.3 million trainable parameters on the edge device, accounting for less than 0.1% of the base model's total parameters.

Datasets. We evaluated all methods on two representative types of downstream tasks. For text classification, we use BoolQ [6], HellaSwag [36], and MMLU [13] datasets. For text generation, we use the Alpaca [30] dataset, which contains instruction-following examples. For all tasks, input prompts are truncated or padded to a uniform context length of 256 tokens.

Baseline Methods. We compare PrivSplit against a comprehensive set of baselines representing different categories of privacy-preserving techniques:

- **Non-private:** The standard distributed PEFT framework without any privacy protection, serving as the utility upper bound.
- **Noise-based Method:** An implementation based on DP-Forward [7] with a privacy budget of $\epsilon = 8$ using the Gaussian Mechanism. It is enhanced with a client-side denoising module as proposed in Split-and-Denoise [20] to create a strong baseline.
- **Permutation-based Method:** An implementation based on SentinelLMs [21], which involves token-level encryption and permutation, followed by a distance-preserving transformation of the embedding space.
- **Anonymization-based Method:** An implementation based on Hide-and-Seek (HaS) [4], which uses a local model to identify and mask or replace sensitive entities in the prompt before sending it to the LLM.

Privacy Attack Setup. To evaluate the privacy protection of each method, we implement three types of attacks. For fair comparison, the specific settings for these attacks are kept consistent with the evaluation in DP-Forward [7].

- **Embedding Inversion Attack (EIA):** We implement a similarity-based attack that performs a nearest-neighbor search to find the token whose embedding is closest to the observed one.

Table 1: Comprehensive results on the privacy-utility trade-off. For each dataset, we report Model Utility (Util. %, higher is better, $\uparrow$) and Attack Success Rates (ASR %, lower is better, $\downarrow$). The ASR column lists three values separated by slashes, corresponding to Embedding Inversion (EIA), Membership Inference (MIA), and Sensitive Attribute Inference (SAIA), respectively.

Method	BoolQ			HellaSwag			MMLU		Alpaca	
	Util. $\uparrow$	ASR $\downarrow$		Util. $\uparrow$	ASR $\downarrow$		Util. $\uparrow$	ASR $\downarrow$	Util. $\uparrow$	ASR $\downarrow$
Non-private	82.1	100.0 / 66.4 / 84.0		75.9	100.0 / 65.9 / 82.7		46.4	100.0 / 65.5 / 80.2	83.9	100.0 / 67.7 / 83.9
Noise-based	76.4	86.2 / 59.4 / 77.1		55.4	80.4 / 50.6 / 71.1		31.0	82.3 / 51.0 / 70.2	75.3	84.6 / 60.4 / 75.5
Permutation	54.7	0.12 / 49.8 / 50.1		29.1	0.09 / 51.1 / 51.6		26.0	0.08 / 50.2 / 51.3	58.2	0.06 / 49.7 / 51.0
Anonymization	74.2	0.11 / 66.2 / 83.8		58.4	0.04 / 65.9 / 82.5		26.8	0.06 / 65.4 / 80.2	76.0	0.09 / 67.7 / 83.7
PrivSplit (Ours)	82.1	0.06 / 49.7 / 49.8		76.0	0.05 / 48.7 / 50.6		46.4	0.01 / 49.9 / 51.0	84.0	0.02 / 50.8 / 50.3

- **Membership Inference Attack (MIA):** We use an entropy-based attack, which assumes that the model’s output distribution for member data will have a lower entropy.
- **Sensitive Attribute Inference Attack (SAIA):** We define a set of entity types as sensitive attributes: DATE, MONEY, PERCENT, QUANTITY, TIME, PERSON, and LANGUAGE. We use the `en_core_web_trf` model from SpaCy [14] to perform named entity recognition on the original prompts to establish a ground truth for the presence of these attributes, against which the success of an attacker’s inference is measured.

Evaluation Metrics. We evaluated the methods in three dimensions. For model utility, we use accuracy for the classification task, while for the generation task, we employ an "LLM as a Judge" [38] approach using GPT-4o [1] to score the semantic equivalence of the output. For privacy protection, we measure the Attack Success Rate (ASR) for each of the three attack types. It is important to note that for the Anonymization-based method, the EIA ASR is calculated only over the tokens identified as sensitive entities, as the method makes no claim to protect the rest of the prompt. For efficiency, we report the end-to-end inference latency in milliseconds, and evaluate the communication overhead by calculating the total volume of data transmitted between the edge and the cloud per inference.

Implementation Details. All experiments were conducted on 2 NVIDIA A40 GPUs. The base model parameters were loaded in float32 precision. For fine-tuning, the AdamW optimizer was used with a learning rate of $1e-4$, a batch size of 8, and all training was run for 3 epochs. For our PrivSplit method, the security parameter σ_{Δ}^2 is set to 100.

4.2 Main Results: Privacy-Utility Trade-off

We present the main results evaluating the trade-off between model utility and prompt privacy in Table 1. The findings clearly demonstrate that PrivSplit achieves strong privacy without compromising model performance.

PrivSplit Achieves Lossless Utility with Strong Privacy. Across all six benchmarks, including classification and generation tasks, PrivSplit’s utility is identical or negligibly different from that of the non-private baseline. This empirically validates our theoretical proof of the method’s lossless nature. Moreover, PrivSplit offers robust privacy protection. Against the most critical threat, the embedding inversion attack, PrivSplit reduces the ASR to near-zero

(0.01-0.06%). Furthermore, it effectively mitigates membership inference and sensitive attribute inference attacks, reducing their ASRs to approximately 50%, which is equivalent to a random guess. This is achieved by setting the variance of the private compensation vectors to a high value ($\sigma_{\Delta}^2 = 100$), a parameter that can be increased for stronger security without any performance penalty.

Comparison with Baselines. PrivSplit’s strength becomes most apparent when contrasted with the trade-offs inherent in other approaches. The noise-based method exhibits a clear privacy-utility compromise, suffering a significant performance drop (e.g., -20.5% on HellaSwag) while still failing to fully prevent inversion (ASR > 80%). An even starker trade-off is seen with the permutation-based method, which achieves excellent privacy protection but at the cost of a catastrophic collapse in model utility (e.g., -46.8% on HellaSwag), demonstrating the critical flaw of breaking model alignment with pre-trained weights. The anonymization-based method offers a different compromise; while it protects specific entities (embedding inversion attack ASR < 0.15%), it shows utility degradation on some tasks and, more importantly, leaves the non-anonymized parts of the prompt completely unprotected, resulting in high MIA and SAIA risk similar to the non-private baseline.

4.3 Computational Overhead Evaluation

In this section, we evaluate the computational overhead introduced by PrivSplit. To precisely isolate the cost of our privacy mechanism, we compare the runtime performance of PrivSplit against the Non-private baseline. We focus our analysis on the Alpaca generation task, as it allows us to measure the overhead on both the initial prompt processing (Prefill) and the subsequent auto-regressive decoding. We report two standard metrics: Time-to-First-Token (TTFT) in milliseconds to measure the initial processing latency, and decoding throughput in tokens per second (tokens/sec) to measure the sustained generation speed.

Table 2: Computational Overhead of PrivSplit on Alpaca with context length 256.

Method	TTFT (ms) $\downarrow$	Throughput (tokens/sec) $\uparrow$
Non-private	501.6	20.0
PrivSplit	508.3	19.8
Overhead (%)	+1.3%	-1.0%

The results presented in Table 2 demonstrate that PrivSplit imposes a negligible and consistent computational overhead. The TTFT, which reflects the cost of the single forward pass on the initial prompt, experiences a marginal increase of only around 1%. This small, fixed cost is incurred during the first-layer compensation and aligns perfectly with our theoretical overhead analysis of $1/(4N)$. Subsequently, as the compensation mechanism must be applied for each new token to ensure losslessness, the decoding throughput also sees a small, consistent decrease.

In summary, the empirical evaluation confirms that PrivSplit's privacy mechanism is highly efficient. The overhead is negligible and consistent across both the initial parallel processing and the subsequent serial generation steps. This demonstrates that PrivSplit provides strong, lossless privacy at an acceptable and negligible computational cost, confirming its practical feasibility for real-world deployment.

4.4 Evaluation of Output Privacy Protection

We experiment to empirically validate the effectiveness of the PrivSplit-OP extension in protecting the generated output. We simulate a powerful cloud adversary that observes the second-to-last layer's output $H_{out}^{(N-1)}$, and attempts to reconstruct the full output sequence by locally applying the base model's final decoder layer and LM Head. To demonstrate the necessity of our design, we compare our full PrivSplit-OP protocol against a Baseline Protocol where the final decoder layer on the edge is not equipped with a private LoRA module.

Table 3: Effectiveness of PrivSplit-OP in preventing output reconstruction attack on the Alpaca dataset.

Method	ASR (%) ↓	Model Utility (%) ↑
Baseline	100.0	84.0
PrivSplit-OP	68.4	84.0

As shown in Table 3, in the insecure baseline protocol, the cloud can perfectly replicate the edge's computation, allowing it to predict the generated token with 100% accuracy. In contrast, under PrivSplit-OP, the cloud's prediction accuracy drops sharply, confirming the effectiveness of our approach. The table also shows that the model's utility remains identical in both scenarios, reaffirming the lossless nature of the protocol. The overhead associated with this extension has been theoretically quantified in Section 3.5.

5 Related Work

5.1 Privacy Attacks on LLM Inputs

The deployment of LLMs in client-server settings has spurred significant research on possible privacy leakage [24]. Although some studies [2, 5, 19] have focused on prompt-crafting attacks to extract sensitive training data such as Personally Identifiable Information (PII), a more fundamental threat lies in the inversion of intermediate representations. Foundational work [17, 22, 28] has demonstrated that a significant portion of original input text can be recovered from internal states like token embeddings. This vulnerability is critically amplified in the distributed PEFT architecture that we

consider. In this setting, the cloud's possession of the public token embedding matrix transforms a difficult optimization-based attack into a trivial lookup-based attack, theoretically allowing for a perfect, 100% inversion of the user's prompt. Alongside direct prompt reconstruction, other documented threats including membership [27] and attribute [10, 25] inference attacks also pose significant risks within this paradigm.

5.2 Privacy-Preserving Methods for LLM Inputs

Existing privacy-preserving methods for LLM input can be broadly classified into three categories, but none are well-suited to the distributed PEFT framework. **Cryptography-based** methods, such as Secure Multi-Party Computation (SMPC) and Homomorphic Encryption (HE) [12, 15, 16, 18], offer strong formal security but incur prohibitive computational and latency overheads, making them impractical for real-time LLM inference [12]. **Noise-based** methods employ DP to add calibrated noise to the input embeddings [7, 9, 20, 29, 31, 34]. While providing formal guarantees, increasing noise inevitably degrades model performance. The utility degradation is particularly severe in deep models like LLMs, where small input perturbations can be progressively amplified through successive layers, leading to significant performance loss that even auxiliary client-side denoising modules [20] have struggled to fully resolve. **Anonymization-based** methods [4, 37] are often implemented as a prompt filter that masks or replaces sensitive information. While practical for well-defined entities like PII, these methods often fail to identify context-dependent sensitive information and can disrupt the prompt's semantic coherence. Recently, SCX [35] was proposed as a lossless method for confidential inference. However, it does not address fine-tuning, and adapting its non-differentiable operations for gradient-based backpropagation would require significant modifications to the protocol. Consequently, it is not applicable to the distributed PEFT framework.

6 Conclusion

This paper addresses the critical privacy vulnerability of user prompts within the distributed PEFT framework. We introduced PrivSplit, a novel protocol founded on a "Split and Compensate" strategy. By decomposing the input embedding on the edge and applying a mathematically exact compensation at the first decoder layer, PrivSplit provides strong privacy guarantees. We formally establish these guarantees by proving its resilience against fundamental inversion and distinguishability attacks. Crucially, we demonstrate that this method is perfectly lossless, maintaining model performance identical to the non-private baseline. Because these guarantees are achieved with only negligible computational overhead and without requiring any cloud-side modifications, PrivSplit paves the way for the practical, secure, and high-performance personalization of Large Language Models in sensitive applications.

Acknowledgments

Lan Zhang is the corresponding author. This research was supported by the China National Natural Science Foundation with No. 62441228, Science and Technology Tackling Program of Anhui Province, No.202423k09020016.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Ulfar Erlingsson, et al. 2021. Extracting training data from large language models. In *30th USENIX security symposium (USENIX Security 21)*. 2633–2650.
- [3] Yuxuan Chen, Rongpeng Li, Zhifeng Zhao, Chenghui Peng, Jianjun Wu, Ekram Hossain, and Honggang Zhang. 2024. NetGPT: An AI-Native Network Architecture for Provisioning Beyond Personalized Generative Services. *IEEE Network* 38, 6 (2024), 404–413. doi:10.1109/MNET.2024.3376419
- [4] Yu Chen, Tingxin Li, Huiming Liu, and Yang Yu. 2023. Hide and seek (has): A light-weight framework for prompt privacy protection. *arXiv preprint arXiv:2309.03057* (2023).
- [5] Shuai Cheng, Shu Meng, Haitao Xu, Haoran Zhang, Shuai Hao, Chuan Yue, Wenrui Ma, Meng Han, Fan Zhang, and Zhao Li. 2025. Effective {PII} Extraction from {LLMs} through Augmented {Few-Shot} Learning. In *34th USENIX Security Symposium (USENIX Security 25)*. 8155–8173.
- [6] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. Boolq: Exploring the surprising difficulty of natural yes/no questions. *arXiv preprint arXiv:1905.10044* (2019).
- [7] Minxin Du, Xiang Yue, Sherman S. M. Chow, Tianhao Wang, Chenyu Huang, and Huan Sun. 2023. DP-Forward: Fine-tuning and Inference on Language Models with Differential Privacy in Forward Pass. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (Copenhagen, Denmark) (CCS '23)*. Association for Computing Machinery, New York, NY, USA, 2665–2679. doi:10.1145/3576915.3616592
- [8] Cynthia Dwork. 2006. Differential privacy. In *International colloquium on automata, languages, and programming*. Springer, 1–12.
- [9] Oluwaseyi Feyisetan, Borja Balle, Thomas Drake, and Tom Diethe. 2020. Privacy- and utility-preserving textual analysis via calibrated multivariate perturbations. In *Proceedings of the 13th international conference on web search and data mining*. 178–186.
- [10] Karan Ganju, Qi Wang, Wei Yang, Carl A Gunter, and Nikita Borisov. 2018. Property inference attacks on fully connected neural networks using permutation invariant representations. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*. 619–633.
- [11] Chao Gao and Sai Qian Zhang. 2024. DLoRA: Distributed Parameter-Efficient Fine-Tuning Solution for Large Language Model. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 13703–13714. doi:10.18653/v1/2024.findings-emnlp.802
- [12] Meng Hao, Hongwei Li, Hanxiao Chen, Pengzhi Xing, Guowen Xu, and Tianwei Zhang. 2022. Iron: Private Inference on Transformers. In *Advances in Neural Information Processing Systems*, Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho (Eds.). https://openreview.net/forum?id=deyqjpcTfsG
- [13] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations*. https://openreview.net/forum?id=d7KBjml3GmQ
- [14] Matthew Honnibal and Ines Montani. 2017. spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing. (2017). To appear.
- [15] Xiaoyang Hou, Jian Liu, Jingyu Li, Yuhua Li, Wen-jie Lu, Cheng Hong, and Kui Ren. 2023. Ciphergpt: Secure two-party gpt inference. *Cryptology ePrint Archive* (2023).
- [16] Dacheng Li, Hongyi Wang, Rulin Shao, Han Guo, Eric Xing, and Hao Zhang. 2023. MPCFORMER: FAST, PERFORMANT AND PRIVATE TRANSFORMER INFERENCE WITH MPC. In *The Eleventh International Conference on Learning Representations*. https://openreview.net/forum?id=CWmvjOEhgH
- [17] Haoran Li, Mingshi Xu, and Yangqiu Song. 2023. Sentence Embedding Leaks More Information than You Expect: Generative Embedding Inversion Attack to Recover the Whole Sentence. In *Findings of the Association for Computational Linguistics: ACL 2023*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 14022–14040. doi:10.18653/v1/2023.findings-acl.881
- [18] Zi Liang, Pinghui Wang, Ruofei Zhang, Nuo Xu, Shuo Zhang, Lifeng Xing, Haitao Bai, and Ziyang Zhou. 2024. Merge: Fast private text generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 19884–19892.
- [19] Nils Lukas, Ahmed Salem, Robert Sim, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2023. Analyzing leakage of personally identifiable information in language models. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 346–363.
- [20] Peihua Mai, Ran Yan, Zhe Huang, Youjia Yang, and Yan Pang. 2024. Split-and-noise: protect large language model inference with local differential privacy. In *Proceedings of the 41st International Conference on Machine Learning (Vienna, Austria) (ICML '24)*. JMLR.org, Article 1395, 22 pages.
- [21] Abhijit Mishra, Mingda Li, and Soham Deo. 2024. Sentinellms: Encrypted input adaptation and fine-tuning of language models for private and secure inference. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 21403–21411.
- [22] John Morris, Volodymyr Kuleshov, Vitaly Shmatikov, and Alexander Rush. 2023. Text Embeddings Reveal (Almost) As Much As Text. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 12448–12460. doi:10.18653/v1/2023.emnlp-main.765
- [23] Mark S Pinsker. 1964. Information and information stability of random variables and processes. *Holden-Day* (1964).
- [24] Yashothara Shanmugarasa, Ming Ding, Chamikara Mahawaga Arachchige, and Thierry Rakotoarivelo. 2025. Sok: The privacy paradox of large language models: Advancements, privacy risks, and mitigation. In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*. 425–441.
- [25] Rakshith Shetty, Bernt Schiele, and Mario Fritz. 2018. {A4NT}: Author attribute anonymity by adversarial training of neural machine translation. In *27th USENIX Security Symposium (USENIX Security 18)*. 1633–1650.
- [26] Haonan Shi, Tu Ouyang, and An Wang. 2025. Navigating the Designs of Privacy-Preserving Fine-tuning for Large Language Models. In *Companion Proceedings of the ACM on Web Conference 2025*. 1298–1302.
- [27] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. 2017. Membership inference attacks against machine learning models. In *2017 IEEE symposium on security and privacy (SP)*. IEEE, 3–18.
- [28] Congzheng Song and Ananth Raghunathan. 2020. Information Leakage in Embedding Models. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (Virtual Event, USA) (CCS '20)*. Association for Computing Machinery, New York, NY, USA, 377–390. doi:10.1145/3372297.3417270
- [29] Jinglin Sun, Basem Suleiman, Imdad Ullah, and Imran Razzak. 2025. Effectiveness of Privacy-preserving Algorithms in LLMs: A Benchmark and Empirical Analysis. In *Proceedings of the ACM on Web Conference 2025*. 5224–5233.
- [30] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An Instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca.
- [31] Meng Tong, Kejiang Chen, Jie Zhang, Yuqi Wang, Weiming Zhang, Nenghai Yu, Tianwei Zhang, and Zhikun Zhang. 2025. Inferdpt: Privacy-preserving inference for black-box large language models. *IEEE Transactions on Dependable and Secure Computing* (2025).
- [32] Hugo Touvron, Thibaut Lavril, Gautier Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [33] Yiming Wang, Yu Lin, Xiaodong Zeng, and Guannan Zhang. 2023. Privatelora for efficient privacy preserving llm. *arXiv preprint arXiv:2311.14030* (2023).
- [34] Zekun Xu, Abhinav Aggarwal, Oluwaseyi Feyisetan, and Nathanael Teissier. 2020. A Differentially Private Text Perturbation Method Using Regularized Mahalanobis Metric. In *Proceedings of the Second Workshop on Privacy in NLP*, Oluwaseyi Feyisetan, Sepideh Ghanavati, Shervin Malmasi, and Patricia Thaine (Eds.). Association for Computational Linguistics, Online, 7–17. doi:10.18653/v1/2020.privatenlp-1.2
- [35] Mu Yuan, Lan Zhang, Liekang Zeng, Siyang Jiang, Bufang Yang, Di Duan, and Guoliang Xing. 2025. SCX: Stateless KV-Cache Encoding for Cloud-Scale Confidential Transformer Serving. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 39–54.
- [36] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830* (2019).
- [37] Ziqian Zeng, Jianwei Wang, Junyao Yang, Zhengdong Lu, Haoran Li, Huiping Zhuang, and Cen Chen. 2025. PrivacyRestore: Privacy-Preserving Inference in Large Language Models via Privacy Removal and Restoration. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 10821–10855. doi:10.18653/v1/2025.acl-long.532
- [38] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.

A The PrivSplit Protocol

Algorithm 1: The PrivSplit Protocol

Input : User's raw text prompt X , number of decoder layers N , set of LoRA layer indices $LoraLayers$.

Output: Generated text response Y .

```

1 /*      Offline Pre-computation      */
2 Edge performs:
3   Pre-compute and store a set of vector pairs  $(\Delta v, h_{p\_in})$ 
   such that for each pair, the condition  $\Delta v^T O_{base}^{(1)} = h_{p\_in}^T$ 
   is met.
4 /*      Online Finetuning / Inference      */
5 // Phase 1: Secure Input Processing
6 Edge performs:
7   Get token embeddings  $H_{in}^{(1)} \leftarrow \text{Embed}(X)$ ;
8   Select a pre-computed pair  $(\Delta v, h_{p\_in})$  from the stored
   set;
9   Construct private part  $H_{p\_in}^{(1)}$  by tiling  $h_{p\_in}$  for all  $L$ 
   tokens;
10  Compute the public part  $H_{e\_in}^{(1)} = H_{in}^{(1)} - H_{p\_in}^{(1)}$ ;
11  Send  $H_{e\_in}^{(1)}$  to cloud.
12 Cloud performs:
13  Compute  $H_e^{(1)} \leftarrow \text{LayerNorm}(H_{e\_in}^{(1)})$ ;
14  Send  $H_e^{(1)}$  to edge;
15 Edge performs:
16  Compute the full hidden state
    $H^{(1)} \leftarrow \text{LayerNorm}(H_{in}^{(1)})$ ;
17  Construct the compensation term  $\Delta V_L$  by tiling the
   selected  $\Delta v$ ;
18  Compute compensated updates  $H_{Q\_edge}^{(1)}, H_{K\_edge}^{(1)}$ 
    $H_{V\_edge}^{(1)}$  according to Eq. (1)(2)(3);
19  Send compensated updates to cloud.
20 Cloud performs:
21  Compute base projections  $H_{Q\_cloud}^{(1)}, H_{K\_cloud}^{(1)}, H_{V\_cloud}^{(1)}$ 
   from  $H_e^{(1)}$  and combine them with compensated
   updates from edge;
22  Execute the rest of decoder layer 1 to get hidden state
    $H_{out}^{(1)}$ .
23 // Phase 2: Standard Distributed PEFT for the
   remainder of Model
24 Cloud & Edge performs:
25   $Y \leftarrow \text{Distributed\_PEFT\_Remainder}(H_{out}^{(1)}, LoraLayers)$ .
```

Here, the superscript (l) denotes the l -th decoder layer, with $H_{in}^{(l)}$, $H^{(l)}$, and $H_{out}^{(l)}$ representing its respective input, post-LayerNorm, and final output hidden states. We assume that the first decoder layer is always an active LoRA layer. For brevity, the function `Execute_Distributed_PEFT_Remainder` abstracts the standard collaborative PEFT process for all subsequent layers.

B Proof & Derivation

B.1 Derivation for Secure Padding Strategy under PrivSplit

This appendix provides the formal derivation for the secure padding strategy mentioned in the main text. The goal is to mitigate the side-channel vulnerability that arises from padded inputs in a batch, where a naive, uniform compensation could reveal patterns to the cloud. The following derivation shows how PrivSplit assigns different compensation vectors to padding and content tokens to break this pattern while maintaining the lossless property.

For a left-padded sequence of total length L with L_0 padding tokens, the combined attention mask (padding mask + causal mask) results in an attention score matrix A_L with a specific block structure:

$$A_L = \begin{pmatrix} \frac{1}{L} J_{L_0} & \frac{1}{L} J \\ O & \tilde{A} \end{pmatrix} \quad (4)$$

Here, J denotes a matrix of ones, O is a zero matrix, and $\tilde{A}$ is the standard causal attention matrix for the $L - L_0$ actual content tokens. Crucially, only the $\tilde{A}$ block is dependent on the input sequence data. Our goal is to find a data-independent compensation matrix ΔV_L that ensures the security of our method against the padding side-channel attack.

To ensure the compensation is data-independent, the term $A_L \Delta V_L$ must be independent of $\tilde{A}$. Formally, for two different inputs that result in different attention matrices A_1 and A_2 , we require the condition $(A_1 - A_2) \Delta V_L W_O = O$. We partition the compensation matrix ΔV_L corresponding to the padding and content sections as $\Delta V_L = \begin{pmatrix} \Delta V_{L_0} \\ \Delta \tilde{V} \end{pmatrix}$. The difference between the attention matrices is non-zero only in the content block:

$$(A_1 - A_2) \Delta V_L = \begin{pmatrix} O & O \\ O & \tilde{A}_1 - \tilde{A}_2 \end{pmatrix} \begin{pmatrix} \Delta V_{L_0} \\ \Delta \tilde{V} \end{pmatrix} = \begin{pmatrix} O & O \\ (\tilde{A}_1 - \tilde{A}_2) \Delta \tilde{V} \end{pmatrix} \quad (5)$$

Therefore, the data-independence condition is satisfied if and only if $(\tilde{A}_1 - \tilde{A}_2) \Delta \tilde{V} = O$. This requires the compensation for the content part, $\Delta \tilde{V}$, to be data-independent, while placing no such constraint on the compensation for the padded part, ΔV_{L_0} .

Based on this derivation, we construct the secure padding for PrivSplit. For the content part, we maintain our original data-independent strategy: $\Delta \tilde{V} = \tilde{\mathbf{1}} \cdot \Delta v^T$, where $\tilde{\mathbf{1}}$ is a vector of those of length $L - L_0$. For the padding part, ΔV_{L_0} , we are free to choose arbitrary row vectors Δv_i^T for $i = 1, \dots, L_0$. To break the side-channel, these vectors can be selected randomly and distinctly from each other and from Δv . This construction results in the following compensated private parts h_{p_t} for each token t :

$$h_{p_t}^T = \frac{1}{L} \left(\sum_{i=1}^{L_0} \Delta v_i^T + (L - L_0) \Delta v^T \right) W_O, \quad t = 1, 2, \dots, L_0 \quad (6)$$

$$h_{p_t}^T = \Delta v^T W_O, \quad t = L_0 + 1, L_0 + 2, \dots, L \quad (7)$$

This approach is provably lossless and successfully mitigates the padding-based side-channel attack by eliminating the observable pattern.

B.2 Proof of Theorem 2 (Input Reconstruction Security)

We prove the theorem by deriving an upper bound on the total information leakage, which is quantified by the mutual information $\epsilon = \mathcal{I}(X; C)$. Our goal is to demonstrate that this leakage is a constant that is small relative to the total entropy $\mathcal{H}(X)$, which validates the tightness of the lower bound on $\mathcal{H}(X|C)$.

The secret input X is processed to produce the normalized hidden state H , which is then used to generate the cloud's observations C . This sequence forms a Markov chain $X \rightarrow H \rightarrow C$. By the data processing inequality, the information leakage about the original input X is upper-bounded by the leakage about the intermediate state H , i.e., $\mathcal{I}(X; C) \leq \mathcal{I}(H; C)$. Therefore, our proof will focus on deriving an upper bound for $\mathcal{I}(H; C)$ and showing that this bound is small compared to the entropy $\mathcal{H}(H)$.

The cloud's observations are $C = (H_{e_in}, H_{Q_edge}, H_{K_edge}, H_{V_edge})$. Using the property of mutual information, the total information leakage can be bounded by the sum of leakages from individual components:

$$\begin{aligned} \mathcal{I}(H; C) &= \mathcal{I}(H; H_{e_in}, H_{Q_edge}, H_{K_edge}, H_{V_edge}) \\ &\leq \mathcal{I}(H; H_{e_in}) + \mathcal{I}(H; H_{Q_edge}) \\ &\quad + \mathcal{I}(H; H_{K_edge}) + \mathcal{I}(H; H_{V_edge}) \end{aligned}$$

We will now bound each term individually.

(1) Bounding Information Leakage from H_{e_in} .

We prove that the information leakage from H_{e_in} vanishes as $\sigma_\Delta^2 \rightarrow \infty$ by establishing a tight lower bound on $\mathcal{H}(H|H_{e_in})$. From the entropy identity $\mathcal{H}(H|H_{e_in}) = \mathcal{H}(H_{in}|H_{e_in}) - \mathcal{H}(H_{in}|H, H_{e_in})$ and the bound $\mathcal{H}(H_{in}|H, H_{e_in}) \leq \mathcal{H}(H_{in}|H)$, we have:

$$\mathcal{H}(H|H_{e_in}) \geq \mathcal{H}(H_{in}|H_{in} - H_p) - \mathcal{H}(H_{in}|H)$$

The term $\mathcal{H}(H_{in}|H) = \sum_{i=1}^L \mathcal{H}(\mu_i, \sigma_i^2)$ represents the information about per-row mean and variance lost during LayerNorm.

The key term is $\mathcal{H}(H_{in}|H_{in} - H_p)$. When the variance of the Gaussian noise H_p (controlled by σ_Δ^2) grows infinitely large, the resulting observation $H_{in} - H_p$ is dominated by noise and reveals vanishingly little information about H_{in} . This is a consequence of limit theorems, provable with characteristic functions, which state that the distribution of this noisy observation converges to a Gaussian. In information-theoretic terms, this means $\mathcal{I}(H_{in}; H_{in} - H_p) \rightarrow 0$, which implies:

$$\lim_{\sigma_\Delta^2 \rightarrow \infty} \mathcal{H}(H_{in}|H_{in} - H_p) = \mathcal{H}(H_{in})$$

Therefore, for a large σ_Δ^2 , the lower bound on the remaining uncertainty is substantial:

$$\mathcal{H}(H|H_{e_in}) \gtrsim \mathcal{H}(H_{in}) - \sum_{i=1}^L \mathcal{H}(\mu_i, \sigma_i^2)$$

Since this bound approaches the high entropy of the pre-normalized input, the information leaked via H_{e_in} is indeed vanishingly small.

(2) Bounding Information Leakage from H_{Q_edge} and H_{K_edge} .

The attacker observes $H_{Q_edge} = H(Q_{lora} + Q_{base}) - H_e Q_{base}$. Since H_e and Q_{base} are public, the attacker can compute an equivalent observation $Y_Q = H(Q_{lora} + Q_{base})$. Let $Q_{sum} = Q_{lora} + Q_{base}$. From the cloud's perspective, the weights Q_{lora} are unknown and

modeled as random. We assume each element of Q_{lora} is drawn from $\mathcal{N}(0, \sigma^2)$, so $\text{vec}(Q_{sum}) \sim \mathcal{N}(\text{vec}(Q_{base}), \sigma^2 \mathbf{I}_{d^2})$. We bound the mutual information $\mathcal{I}(H; Y_Q) = \mathcal{H}(Y_Q) - \mathcal{H}(Y_Q|H)$.

First, we compute the conditional entropy $\mathcal{H}(Y_Q|H)$. Given a fixed H , Y_Q is a linear transformation of the Gaussian variable Q_{sum} . Thus, $Y_Q|H$ is also Gaussian. Conditional on H , the distribution is:

$$\text{vec}(Y_Q)|H \sim \mathcal{N}((\mathbf{I}_d \otimes H)\text{vec}(Q_{base}), \sigma^2(\mathbf{I}_d \otimes (HH^T)))$$

The differential entropy of a k -dimensional multivariate normal distribution $\mathcal{N}(\mu, \Sigma)$ is $\frac{1}{2} \log((2\pi e)^k \det(\Sigma))$. Here, the dimension is dL :

$$\begin{aligned} \mathcal{H}(Y_Q|H) &= \frac{1}{2} \log((2\pi e)^{dL} \det(\sigma^2 \mathbf{I}_d \otimes (HH^T))) \\ &= \frac{dL}{2} \log(2\pi e \sigma^2) + \frac{d}{2} \log(\det(HH^T)) \end{aligned}$$

Next, we upper-bound $\mathcal{H}(Y_Q)$. By the maximum entropy principle, the entropy of Y_Q is upper-bounded by the entropy of a Gaussian variable with the same covariance matrix. Assuming the elements of H are i.i.d. with distribution $\mathcal{N}(0, \sigma_H^2)$ to find an upper bound. Let λ_{Q_i} be the eigenvalues of $Q_{base}^T Q_{base}$. The entropy of this Gaussian distribution is:

$$\begin{aligned} \mathcal{H}_{Gauss}(Y_Q) &= \frac{1}{2} \log \det(2\pi e \text{Cov}(\text{vec}(Y_Q))) \\ &= \frac{dL}{2} \log(2\pi e \sigma_H^2) + \frac{L}{2} \sum_{i=1}^d \log(d\sigma^2 + \lambda_{Q_i}) \end{aligned}$$

Thus, $\mathcal{H}(Y_Q) \leq \mathcal{H}_{Gauss}(Y_Q)$. The mutual information is bounded by:

$$\begin{aligned} \mathcal{I}(H; Y_Q) &\leq \mathcal{H}_{Gauss}(Y_Q) - \mathbb{E}_H[\mathcal{H}(Y_Q|H)] \\ &= \frac{dL}{2} \log(\sigma_H^2) + \frac{L}{2} \sum_{i=1}^d \log(d + \frac{\lambda_{Q_i}}{\sigma^2}) \\ &\quad - \frac{d}{2} \mathbb{E}_H[\log(\det(HH^T))] \end{aligned}$$

The analysis for H_{K_edge} is identical to that for H_{Q_edge} , resulting in a similarly bounded mutual information term $\mathcal{I}(H; Y_K)$.

(3) Bounding Information Leakage from H_{V_edge} .

For $H_{V_edge} = H(V_{lora} + V_{base}) - H_e V_{base} + \Delta V_L$, we note the addition of an independent noise term ΔV_L . Let $H'_{V_edge} = H(V_{lora} + V_{base}) - H_e V_{base}$. The variables form a Markov chain $H \rightarrow H'_{V_edge} \rightarrow H_{V_edge}$. By the Data Processing Inequality, we have: $\mathcal{I}(H; H_{V_edge}) \leq \mathcal{I}(H; H'_{V_edge})$. The bound for $\mathcal{I}(H; H'_{V_edge})$ is derived in the same way as for H_{Q_edge} . Therefore, the leakage from H_{V_edge} is also bounded.

Summing these individual upper bounds, we conclude that the information leakage from H_{e_in} vanishes as the security parameter σ_Δ^2 increases, while the leakage from each edge component is bounded by a fixed constant. This completes the proof of Theorem 2.

NOTE. We numerically validate our bounds' tightness using LLaMA2-7B parameters ($d = 4096, L = 512$). The total secret entropy is estimated as $\mathcal{H}(H) \approx 1.09 \times 10^7$ nats, while the total information leakage is bounded by $\epsilon \leq 2.08 \times 10^5$ nats. This leakage accounts for less than 1.9% of the total entropy, thus providing strong numerical evidence for our theoretical claims. $\square$

B.3 Proof of Theorem 3 (Input Distinguishability Security)

Our strategy is to show that the statistical distance between the probability distributions of the cloud's observations for two distinct inputs, $P_{C(0)}$ and $P_{C(1)}$, converges to zero as $\sigma_\Delta^2 \rightarrow \infty$.

(1) From Attacker's Advantage to Total Variation Distance

The advantage of any attacker $\mathcal{A}$ in distinguishing between the two cases ($b = 0$ and $b = 1$) is defined as:

$$\text{Adv}(\mathcal{A}) = \left| \Pr[\mathcal{A}(C) = b|b] - \frac{1}{2} \right|$$

For an optimal attacker, this advantage is directly related to the Total Variation Distance (TVD) between the two distributions:

$$\text{Adv}^*(\mathcal{A}) = \max_{\mathcal{A}} \text{Adv}(\mathcal{A}) = \frac{1}{2} \delta(P_{C(0)}, P_{C(1)})$$

where the TVD is defined as $\delta(P, Q) = \frac{1}{2} \int |p(x) - q(x)| dx$. To prove the theorem, we need to show that $\delta(P_{C(0)}, P_{C(1)}) \rightarrow 0$.

(2) From Total Variation Distance to KL Divergence

Pinsker's inequality [23] provides an upper bound on the TVD in terms of the Kullback-Leibler (KL) divergence:

$$\delta(P, Q) \leq \sqrt{\frac{1}{2} D_{KL}(P||Q)}$$

Thus, our goal is reduced to proving that $D_{KL}(P_{C(0)}||P_{C(1)}) \rightarrow 0$ as $\sigma_\Delta^2 \rightarrow \infty$.

(3) Bounding the KL Divergence

Let $C = (H_{e_{in}}, Y)$, where $Y = (Y_Q, Y_K, Y_V)$ represents the set of equivalent observations for the *edge* components. Using the chain rule for KL divergence:

$$\begin{aligned} D_{KL}(P_{C(0)}||P_{C(1)}) &= D_{KL}(P_{H_{e_{in}}^{(0)}}||P_{H_{e_{in}}^{(1)}}) \\ &\quad + \mathbb{E}_{H_{e_{in}} \sim P_{H_{e_{in}}^{(0)}}} \left[D_{KL}(P_{Y|H_{e_{in}}^{(0)}}||P_{Y|H_{e_{in}}^{(1)}}) \right] \end{aligned}$$

We will show that both terms on the right-hand side converge to zero as $\sigma_\Delta^2 \rightarrow \infty$.

The observation $H_{e_{in}}^{(b)}$ is the result of subtracting a Gaussian noise matrix H_p from the input dependent matrix $H_{in}^{(b)}$. Specifically, $H_{e_{in}}^{(b)} = H_{in}^{(b)} - H_p$. Each row of H_p is an identical sample from $\mathcal{N}(\mathbf{0}, \Sigma_p)$, where $\Sigma_p = \sigma_\Delta^2 W_O^T W_O$.

Let's vectorize the matrices for a standard KL divergence formulation. Let $\mathbf{v}_{e_{in}}^{(b)} = \text{vec}((H_{e_{in}}^{(b)})^T)$. The distribution of $\mathbf{v}_{e_{in}}^{(b)}$ is a multivariate Gaussian $\mathcal{N}(\boldsymbol{\mu}_b, \Sigma)$, where:

- The mean is $\boldsymbol{\mu}_b = \text{vec}((H_{in}^{(b)})^T)$.
- The covariance matrix Σ is independent of the input b and is given by $\text{Cov}(\text{vec}(H_p^T)) = J_L \otimes \Sigma_p$, where J_L is the $L \times L$ matrix of all ones.

The KL divergence between two multivariate normal distributions $\mathcal{N}(\boldsymbol{\mu}_0, \Sigma)$ and $\mathcal{N}(\boldsymbol{\mu}_1, \Sigma)$ with the same covariance is:

$$D_{KL}(P_0||P_1) = \frac{1}{2} (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)^T \Sigma^{-1} (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)$$

Since $\Sigma = J_L \otimes \Sigma_p$ is singular, we use the Moore-Penrose pseudoinverse: $\Sigma^\dagger = (J_L \otimes \Sigma_p)^\dagger = J_L^\dagger \otimes \Sigma_p^\dagger$. We have $J_L^\dagger = \frac{1}{L^2} J_L$ and $\Sigma_p^\dagger = (\sigma_\Delta^2 W_O^T W_O)^{-1} = \frac{1}{\sigma_\Delta^2} (W_O^T W_O)^{-1}$. So, $\Sigma^\dagger = \frac{1}{L^2 \sigma_\Delta^2} J_L \otimes (W_O^T W_O)^{-1}$.

Let $\Delta\boldsymbol{\mu} = \boldsymbol{\mu}_1 - \boldsymbol{\mu}_0 = \text{vec}((H_{in}^{(1)} - H_{in}^{(0)})^T)$. Let $\overline{\Delta\mathbf{h}_{in}}$ be the mean row difference vector between $H_{in}^{(1)}$ and $H_{in}^{(0)}$. The quadratic form simplifies to:

$$(\Delta\boldsymbol{\mu})^T (J_L \otimes (W_O^T W_O)^{-1}) (\Delta\boldsymbol{\mu}) = L^2 (\overline{\Delta\mathbf{h}_{in}})^T (W_O^T W_O)^{-1} (\overline{\Delta\mathbf{h}_{in}})$$

Substituting this back into the KL divergence formula:

$$\begin{aligned} D_{KL}(P_{H_{e_{in}}^{(0)}}||P_{H_{e_{in}}^{(1)}}) &= \frac{1}{2} \cdot \frac{1}{L^2 \sigma_\Delta^2} \cdot [L^2 (\overline{\Delta\mathbf{h}_{in}})^T (W_O^T W_O)^{-1} (\overline{\Delta\mathbf{h}_{in}})] \\ &= \frac{1}{2 \sigma_\Delta^2} (\overline{\Delta\mathbf{h}_{in}})^T (W_O^T W_O)^{-1} (\overline{\Delta\mathbf{h}_{in}}) \end{aligned}$$

where $(\overline{\Delta\mathbf{h}_{in}})^T (W_O^T W_O)^{-1} (\overline{\Delta\mathbf{h}_{in}})$ is a constant that depends on the inputs but not on σ_Δ^2 . As $\sigma_\Delta^2 \rightarrow \infty$, it is clear that:

$$\lim_{\sigma_\Delta^2 \rightarrow \infty} D_{KL}(P_{H_{e_{in}}^{(0)}}||P_{H_{e_{in}}^{(1)}}) = 0$$

Next, we analyze the conditional distribution $P(Y|H_{e_{in}}, b)$. The observations $Y = (Y_Q, Y_K, Y_V)$ are functions of the normalized hidden state $H = \text{LayerNorm}(H_{in}^{(b)} + H_p)$. Given $H_{e_{in}}^{(b)} = H_{in}^{(b)} - H_p$, we can write $H = \text{LayerNorm}(H_{e_{in}}^{(b)} + H_p)$.

As the security parameter $\sigma_\Delta^2 \rightarrow \infty$, the variance of the elements of the noise matrix H_p grows infinitely large. In the sum $H_{e_{in}}^{(b)} + H_p$, the contribution of the fixed-magnitude matrix $H_{e_{in}}^{(b)}$ becomes negligible compared to the high-variance noise H_p . The statistical properties of the sum become dominated entirely by H_p . Since H_p is independent of the input index b , the distribution of the sum $H_{e_{in}}^{(b)} + H_p$ converges to the distribution of H_p .

Consequently, the distribution of $H = \text{LayerNorm}(H_{e_{in}}^{(b)} + H_p)$ also converges to a distribution that is independent of b . Since Y is a function of H (and other random variables independent of b), the conditional distribution $P(Y|H_{e_{in}}, b)$ must converge to a limiting distribution that does not depend on b .

Therefore, for any fixed $H_{e_{in}}$, we have

$$\lim_{\sigma_\Delta^2 \rightarrow \infty} P_{Y|H_{e_{in}}^{(0)}} = \lim_{\sigma_\Delta^2 \rightarrow \infty} P_{Y|H_{e_{in}}^{(1)}}$$

This implies that the KL divergence between them converges pointwise to zero:

$$\lim_{\sigma_\Delta^2 \rightarrow \infty} D_{KL}(P_{Y|H_{e_{in}}^{(0)}}||P_{Y|H_{e_{in}}^{(1)}}) = 0$$

Since the KL divergence is non-negative and converges to 0 for every value in its domain, by the Dominated Convergence Theorem, the expectation of this KL divergence must also converge to 0.

Now we have shown that both terms in the chain rule expansion of $D_{KL}(P_{C(0)}||P_{C(1)})$ converge to zero as $\sigma_\Delta^2 \rightarrow \infty$. Therefore,

$$\lim_{\sigma_\Delta^2 \rightarrow \infty} D_{KL}(P_{C(0)}||P_{C(1)}) = 0$$

By Pinsker's inequality, this implies that $\lim_{\sigma_\Delta^2 \rightarrow \infty} \delta(P_{C(0)}, P_{C(1)}) = 0$. Finally, this means the optimal attacker's advantage vanishes:

$$\lim_{\sigma_\Delta^2 \rightarrow \infty} \text{Adv}(\mathcal{A}) \leq \lim_{\sigma_\Delta^2 \rightarrow \infty} \text{Adv}^*(\mathcal{A}) = \lim_{\sigma_\Delta^2 \rightarrow \infty} \frac{1}{2} \delta(P_{C(0)}, P_{C(1)}) = 0$$

This completes the proof of Theorem 3.